Python Activity-Based Learning
Case Study

29 Password Policy Validation Engine

Selected case study | Prepared using the topics listed in the supplied assignment.

1. Problem Statement

Validate sample passwords using minimum length, uppercase, lowercase, digit, and special-character requirements. Report which rules are not satisfied.

2. Objectives

• Process strings using loops.

• Check character categories using string methods.

• Use conditions to validate password rules.

• Display individual validation results.

3. Assumptions

• The minimum password length is 8 characters.

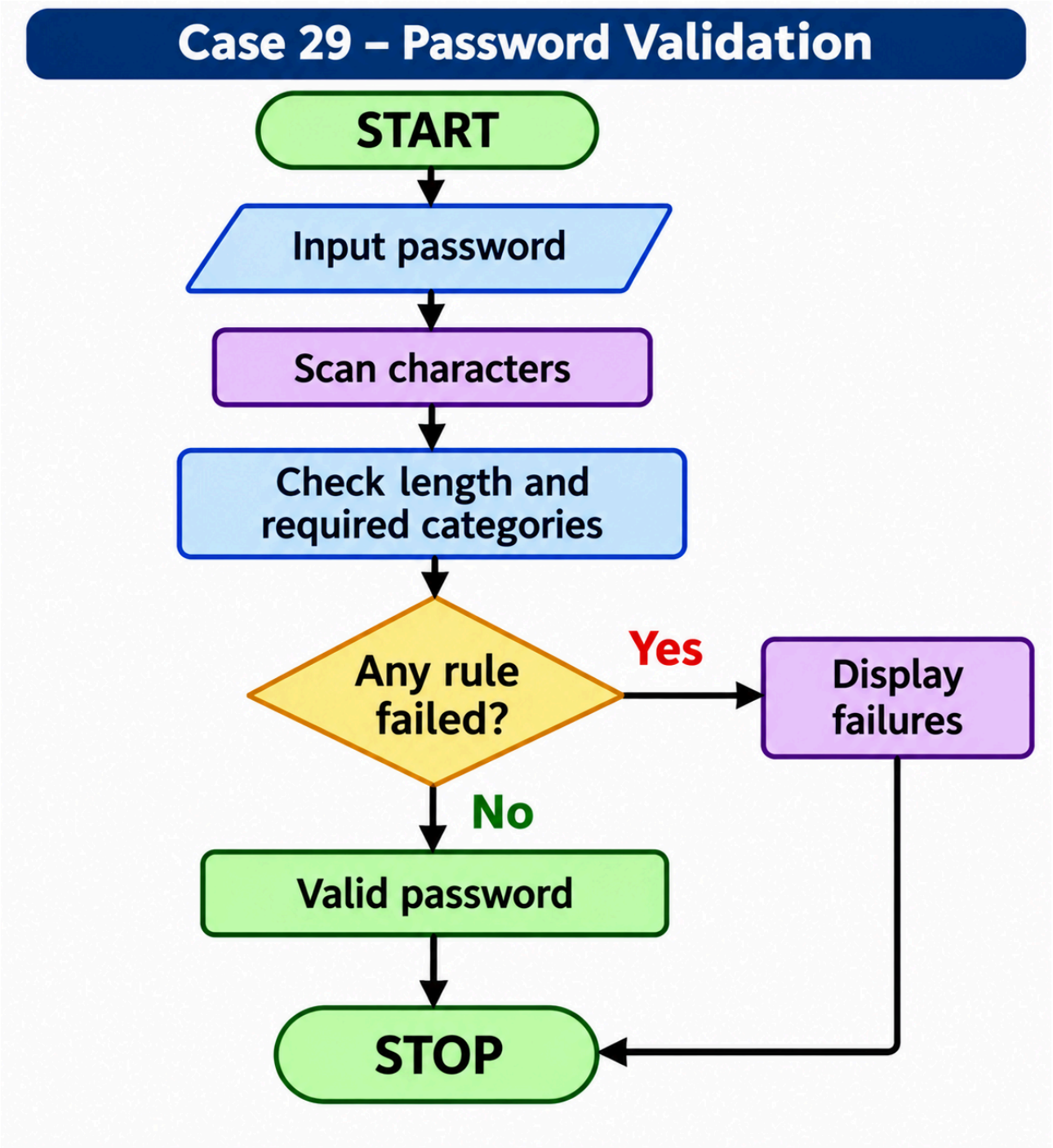
• A valid password must contain at least one uppercase letter, lowercase letter, digit, and special character.

• Spaces are not accepted.

4. Algorithm

1. Read a password.
2. Initialize flags for uppercase, lowercase, digit, and special character.
3. Scan each character and update the appropriate flags.
4. Check length and spaces.
5. Display each failed rule.
6. If no rule fails, report that the password is valid.

5. Flowchart



6. Pseudocode

```
BEGIN
 INPUT password
 Set upper, lower, digit, special flags to FALSE
 FOR each character in password
 Update matching category flag
 END FOR
```

```
 Check length >= 8 and no spaces
 Display failed rules
 IF all rules pass THEN PRINT VALID
 ELSE PRINT INVALID
END
```

Case Study 29 — Password Policy Validation Engine

7. Python Source Code

```
password = input("Enter password: ")

has_upper = False
has_lower = False
has_digit = False
has_special = False

for ch in password:
 if ch.isupper():
 has_upper = True
 elif ch.islower():
 has_lower = True
 elif ch.isdigit():
 has_digit = True
 elif not ch.isspace():
 has_special = True

valid = True

if len(password) < 8:
 print("Failed: minimum 8 characters required")
 valid = False
if not has_upper:
 print("Failed: include an uppercase letter")
 valid = False
if not has_lower:
 print("Failed: include a lowercase letter")
 valid = False
if not has_digit:
 print("Failed: include a digit")
 valid = False
if not has_special:
 print("Failed: include a special character")
 valid = False
if " " in password:
 print("Failed: spaces are not allowed")
 valid = False

if valid:
 print("Password is valid")
else:
 print("Password is invalid")
```

8. Test Cases

Input / Scenario	Expected result
Abcdef1@	Valid
abcdef1@	Fails uppercase rule
ABC12345	Fails lowercase and special-character rules

9. Conclusion

The program demonstrates string handling, character testing, loops, flags, and conditional validation. Note:

Run the program in Python 3. The sample test cases are illustrative; enter the values when prompted.